

Exercise A: For the following tasks use the table **EMPLOYEES**

1. Display the first name, the last name and the salary for Abel.

```
SELECT first_name, last_name, salary  
FROM EMPLOYEES  
WHERE last_name = 'Abel';
```

2. Arrange all the employees using salary and display the first_name, the last_name and the salary.

```
SELECT first_name, last_name, salary  
FROM EMPLOYEES  
ORDER BY salary;
```

3. Decrease the salary by 200 and display the first name, the last name, the old salary and the new salary for employees having department id=90.

```
SELECT first_name, last_name, salary-200, salary  
FROM EMPLOYEES  
WHERE department_id=90;
```

4. Display the average salary for employees where department_id = 90.

```
SELECT avg(salary)  
FROM EMPLOYEES  
WHERE department_id=90;
```

5. Display the average salary for employees where job_id = it_prog and round to two decimal places.

```
SELECT ROUND(avg(salary),2)  
FROM EMPLOYEES  
WHERE job_id = 'IT_PROG';
```

6. Display the latest hire date for department_id=90.

```
SELECT MAX(hire_date)  
FROM EMPLOYEES  
WHERE department_id = 90;
```

7. Display the last name in the alphabetic list of names.

```
SELECT last_name  
FROM EMPLOYEES  
ORDER BY last_name ASC;
```

8. Display the first in the alphabetic list of names name for employees having department_id = 90.

```
SELECT last_name  
FROM EMPLOYEES  
WHERE department_id = 90  
ORDER BY last_name ASC;
```

9. Display the lowest salary from employees having manager_id= 100.

```
SELECT MIN(SALARY)  
FROM EMPLOYEES  
WHERE manager_id= 100;
```

10. Display all the information about employees having last_name starting with „K”.

```
SELECT *  
FROM EMPLOYEES  
WHERE last_name like 'K%';
```

11. Display first_name and last_name in a new column “Name” from employees having job_id = sa_rep.

```
SELECT first_name || ' ' || last_name as "Name"  
FROM EMPLOYEES  
WHERE job_id = 'SA_REP';
```

12. Display the last_name with capital letters for employees having department_id= 50 or 80.

```
SELECT UPPER(last_name)  
FROM EMPLOYEES  
WHERE department_id in (50, 80);
```

13. Display the first_name, the last_name, and the salary for employees who earn more than Ernst. Order by last_name.

```
SELECT first_name, last_name, salary  
FROM EMPLOYEES  
WHERE salary > (SELECT salary FROM EMPLOYEES WHERE last_name = 'Ernst')  
ORDER BY last_name;
```

14. Display all the information for employees who work in the same department as Davies.

```
SELECT *  
FROM EMPLOYEES  
WHERE department_id = (SELECT department_id FROM EMPLOYEES WHERE last_name =  
'Davies');
```

15. Display all the information for employees who earn less than Lorentz and are in the same department as Davies.

```
SELECT *  
FROM EMPLOYEES  
WHERE department_id = (SELECT department_id FROM EMPLOYEES WHERE last_name =  
'Davies') AND salary < (SELECT salary FROM EMPLOYEES WHERE last_name = 'Lorentz');
```

16. Display the first name, the last name and the hire date for employees who have the same job_id as Rajs.

```
SELECT first_name, last_name, hire_date  
FROM EMPLOYEES  
WHERE job_id = (SELECT job_id FROM EMPLOYEES WHERE last_name = 'Rajs');
```

17. Display the first name, the last name and the hire date for employees hired after Davies.

```
SELECT first_name, last_name, hire_date  
FROM EMPLOYEES  
WHERE hire_date > (SELECT hire_date FROM EMPLOYEES WHERE last_name = 'Davies');
```

18. Display the first name, the last name and the hire date for employees who have the same job id as Rajs and were hired after De Haan.

```
SELECT first_name, last_name, hire_date  
FROM EMPLOYEES  
WHERE job_id = (SELECT job_id FROM EMPLOYEES WHERE last_name = 'Rajs' and hire_date >  
(SELECT hire_date FROM EMPLOYEES WHERE last_name = 'De Haan'));
```

1. Arrange all the employees using hire_date, and display first_name, last_name and the location_id from employees who earn more than Davies.

```
SELECT first_name, last_name, location_id  
FROM EMPLOYEES NATURAL JOIN Departments  
WHERE salary > (SELECT salary FROM EMPLOYEES WHERE last_name = 'Davies')  
ORDER BY hire_date;
```

2. Display all the information for Davies' manager.

```
SELECT *  
FROM EMPLOYEES  
WHERE employee_id = (SELECT manager_id FROM EMPLOYEES WHERE last_name = 'Davies');
```

3. Display the first_name, the last_name, and the salary for Davies' manager.

```
SELECT first_name, last_name, salary  
FROM EMPLOYEES  
WHERE employee_id = (SELECT manager_id FROM EMPLOYEES WHERE last_name = 'Davies');
```

4. Arrange all the employees having the same department as Matos using salary, and display first_name, last_name and the salary.

```
SELECT first_name, last_name, salary
FROM EMPLOYEES
WHERE department_id = (SELECT department_id FROM EMPLOYEES WHERE last_name =
'Matos')
ORDER BY salary;
```

5. Display the first_name, the last_name, and the salary for employees who earns more than the minimum salaries for all the employees having department_id=80.

```
SELECT first_name, last_name, salary
FROM EMPLOYEES
WHERE salary > (SELECT min(salary) FROM EMPLOYEES WHERE department_id=80);
```

6. Arrange all the employees having the same department as Abel using salary, and display first_name, last_name and the salary.

```
SELECT first_name, last_name, salary
FROM EMPLOYEES
WHERE department_id = (SELECT department_id FROM EMPLOYEES WHERE first_name =
'Abel')
ORDER BY salary;
```

7. Arrange all the employees hired after Abel using salary, and display first_name, last_name, salary and the hire_date.

```
SELECT first_name, last_name, salary, hire_date
FROM EMPLOYEES
WHERE hire_date > (SELECT hire_date FROM EMPLOYEES WHERE first_name = 'Abel')
ORDER BY salary;
```

Exercise B: For the following tasks use the tables: **EMPLOYEES, DEPARTMENTS, LOCATIONS, JOB_HISTORY.**

1. Display the first_name, the last_name, the salary and the hire_date (from employees) and the department_name and the location_id (from departments).

```
SELECT first_name, last_name, salary, hire_date, department_name, location_id  
FROM EMPLOYEES NATURAL JOIN DEPARTMENTS;
```

2. Display the first_name, the last_name, the salary, the department_id and the hire_date (from employees) and the department_name and the location_id (from departments) for employees having salary > 8000.

```
SELECT first_name, last_name, salary, hire_date, department_name, location_id  
FROM EMPLOYEES NATURAL JOIN DEPARTMENTS  
WHERE salary > 8000;
```

3. Display the first_name, the last_name, the salary and the hire_date (from employees) and department_name and location_id (from departments) for employees who earn more than Ernst. Order by last_name.

```
SELECT first_name, last_name, salary, hire_date, department_name, location_id  
FROM EMPLOYEES NATURAL JOIN DEPARTMENTS  
WHERE salary > (SELECT salary FROM EMPLOYEES WHERE first_name = 'Ernst')  
ORDER BY last_name;
```

4. Display the first_name, the last_name, the salary (from employees), the department_name (from departments) the street_address and the city (from locations).

```
SELECT first_name, last_name, salary, department_name, street_address  
FROM EMPLOYEES NATURAL JOIN DEPARTMENTS NATURAL JOIN LOCATIONS
```

5. Display the first_name, the last_name, the salary (from employees), the department_name (from departments) the street_address and the city (from locations) for employees who earn more than Grant. Order by salary.

```
SELECT first_name, last_name, salary, department_name, street_address, city  
FROM EMPLOYEES JOIN DEPARTMENTS using(department_id) JOIN LOCATIONS USING  
(location_id)  
WHERE salary > (SELECT salary FROM EMPLOYEES WHERE first_name = 'Grant')  
ORDER BY salary;
```

6. Display the salary, bonus, first_name and last_name (from employees) and department_name and manager_id (from departments)

```
SELECT salary, bonus, first_name, last_name, department_name, manager_id  
FROM EMPLOYEES NATURAL JOIN DEPARTMENTS;
```

7. Display the salary, bonus, first_name and last_name(from employees) and department_name and manager_id(from departments) where manager_id=124.

```
SELECT salary, bonus, first_name, last_name, department_name, manager_id  
FROM EMPLOYEES Natural JOIN DEPARTMENTS  
WHERE manager_id=124;
```

1. Display the salary, bonus, first_name and last_name(from employees) and department_name and manager_id(from departments) where salary> 10000.

```
SELECT salary, bonus, first_name, last_name, department_name, manager_id  
FROM EMPLOYEES Natural JOIN DEPARTMENTS  
WHERE salary> 10000;
```

2. Display first_name and last_name(from employees) and location_id(from departments) from employees hired after 01.09.1990.

```
SELECT first_name, last_name, location_id  
FROM EMPLOYEES Natural JOIN DEPARTMENTS  
WHERE hire_date > '01-Sep-1990';
```

3. Display the first_name, the last_name, the salary(from employees) and department_name and location_id(from departments) for employees who earn less than the average of the salaries for employees having department_id =50 or 90.

```
SELECT first_name, last_name, salary, location_id, department_name  
FROM EMPLOYEES Natural JOIN DEPARTMENTS  
WHERE salary < (SELECT AVG(salary) FROM EMPLOYEES WHERE department_id IN (50, 90));
```

4. Display the first_name, the last_name and the salary(from employees), the department_name, and the location_id(from departments) from employees having the same department_name as Matos.

5. Increase the salary with 500 and display the first_name, the last_name, the salary, the new salary(from employees) and the location_id(from departments) for employees hired after Davies.

```
SELECT first_name, last_name, salary, salary + 500, location_id  
FROM EMPLOYEES NATURAL JOIN DEPARTMENTS  
WHERE hire_date > (SELECT hire_date FROM EMPLOYEES WHERE last_name = 'Davies');
```

6. Create a join between the tables EMPLOYEES and DEPARTMENTS and display all the information from both tables for employees having department_id=90.

```
SELECT *  
FROM EMPLOYEES NATURAL JOIN DEPARTMENTS  
WHERE department_id = 90;
```

7. Create a natural join between the tables EMPLOYEES and JOB_HISTORY and display all the information from both tables.

```
SELECT *  
FROM EMPLOYEES NATURAL JOIN JOB_HISTORY;
```

8. Create a join between the tables EMPLOYEES and JOB_HISTORY using department_id and display the first_name, last_name and salary from employees and start_date and end_date from job_history.

```
SELECT first_name, last_name, salary, start_date, end_date  
FROM EMPLOYEES JOIN JOB_HISTORY USING (department_id);
```

9. Create a join between the tables EMPLOYEES and JOB_HISTORY using department_id and display the first_name, last_name and salary from employees and start_date and end_date from job_history for employees having first_name starting with „M”.

```
SELECT first_name, last_name, salary, start_date, end_date, job_history  
FROM EMPLOYEES JOIN JOB_HISTORY USING (department_id)  
WHERE first_name like 'M%';
```

10. Create a join between DEPARTMENTS and LOCATIONS and display the department_id, the department_name, the street_address, the city, and the postal_code for the employees having the same department_id as Matos.

```
SELECT department_id, department_name, street_address, city, postal_code  
FROM DEPARTMENTS NATURAL JOIN LOCATIONS  
WHERE department_id = (SELECT department_id FROM EMPLOYEES WHERE last_name like  
'Matos');
```

11. Create a join between EMPLOYEES, DEPARTMENTS and LOCATIONS and display the first_name, the last_name, the salary, the manager_id, the department_name, the location_id, the street_address, and the city for all the employees having the same manager as Rajs.

```
SELECT first_name, last_name, salary, manager_id, department_name, location_id,  
street_address, city  
FROM EMPLOYEES NATURAL JOIN DEPARTMENTS NATURAL JOIN LOCATIONS  
WHERE manager_id = (SELECT manager_id FROM EMPLOYEES WHERE first_name = 'Rajs');
```

12. Create a join between DEPARTMENTS and EMPLOYEES and display the first_name, the last_name, the salary, the department_name and the location_id for all the employees having the same manager as Matos. Arrange in descending order using the salary.

```
SELECT first_name, last_name, salary, department_name, location_id
FROM EMPLOYEES NATURAL JOIN DEPARTMENTS
WHERE manager_id = (SELECT manager_id FROM EMPLOYEES WHERE first_name = 'Matos')
ORDER BY salary DESC;
```

1 Create a cross join between the tables EMPLOYEES and JOB_HISTORY and display all the information from both tables.

```
SELECT *
FROM EMPLOYEES
CROSS JOIN JOB_HISTORY;
```

2 Create a natural join between the tables EMPLOYEES and JOB_HISTORY and display the first_name, last_name, the department_id, the start_date and the end_date.

```
SELECT first_name, last_name, department_id, start_date, end_date
FROM EMPLOYEES NATURAL JOIN JOB_HISTORY;
```

3 Create a natural join between the tables EMPLOYEES and JOB_HISTORY and display the first_name, last_name, the department_id, the start_date and the end_date.

```
SELECT first_name, last_name, department_id, start_date, end_date
FROM EMPLOYEES NATURAL JOIN JOB_HISTORY;
```

4 Create a natural join between the tables EMPLOYEES and JOB_HISTORY and display the first_name, last_name from employees and start_date and end_date from job_history.

```
SELECT first_name, last_name, start_date, end_date
FROM EMPLOYEES NATURAL JOIN JOB_HISTORY;
```

5 Create a join between the tables EMPLOYEES and JOB_HISTORY using employee_id and display the first_name, last_name, the employee_id, the start_date and the end_date.

```
SELECT first_name, last_name, employee_id, start_date, end_date
FROM EMPLOYEES JOIN JOB_HISTORY USING (employee_id);
```

6 Create a cross join between the tables DEPARTMENTS and JOB_HISTORY and display all the information from both tables.

```
SELECT *
FROM DEPARTMENTS CROSS JOIN JOB_HISTORY;
```

7 Create a natural join between the tables DEPARTMENTS and JOB_HISTORY and display the department_name, location_id from departments and start_date and end_date from job_history.

```
SELECT department_name, location_id, start_date, end_date  
FROM DEPARTMENTS NATURAL JOIN JOB_HISTORY;
```

8 Create a join between the tables DEPARTMENTS and JOB_HISTORY using department_id and display the department_name, location_id, the start_date and the end_date.

```
SELECT department_name, location_id, start_date, end_date  
FROM DEPARTMENTS JOIN JOB_HISTORY USING (department_id);
```

9 Create a natural join between the tables EMPLOYEES and DEPARTMENTS and display all the information from both tables for employees having department_id=80.

```
SELECT *  
FROM EMPLOYEES NATURAL JOIN DEPARTMENTS  
WHERE department_id = 80;
```

10 Make a join between EMPLOYEES and DEPARTMENTS and display first_name, last_name, salary, hire_date, department_name and manager_id.

```
SELECT first_name, last_name, salary, hire_date, department_name, manager_id  
FROM EMPLOYEES NATURAL JOIN DEPARTMENTS;
```

11 Create a join between EMPLOYEES and DEPARTMENTS and display all the information about employees who work in the same department as Matos. Arrange all the employees using the hire_date.

```
SELECT *  
FROM EMPLOYEES NATURAL JOIN DEPARTMENTS  
where department_id = (SELECT department_id FROM EMPLOYEES WHERE last_name like  
'Matos' );
```

1. Create a join between the tables EMPLOYEES and JOB_HISTORY and display the first_name, last_name, salary, start_date, end_date from employees having department_id=50.

```
SELECT first_name, last_name, salary, start_date, end_date  
FROM EMPLOYEES NATURAL JOIN JOB_HISTORY  
where department_id = 50;
```

Exercise C: For the following tasks use the tables: **REGIONS** and **COUNTRIES**

1. Create a cross join between the tables REGIONS and COUNTRIES and display all the information.

```
SELECT *  
FROM REGIONS CROSS JOIN COUNTRIES;
```

2. Create a join between the tables REGIONS and COUNTRIES using region_id and display all the information.

```
SELECT *  
FROM REGIONS JOIN COUNTRIES USING(region_id);
```

3. Create a join between the tables REGIONS and COUNTRIES using region_id and display all the information about the Canada.

```
SELECT *  
FROM REGIONS JOIN COUNTRIES USING(region_id)  
WHERE COUNTRY_NAME = 'Canada';
```

4. Create a join between the tables REGIONS and COUNTRIES using region_id and display all the information about the countries having region_id=2.

```
SELECT *  
FROM REGIONS JOIN COUNTRIES USING(region_id)  
WHERE region_id=2;
```